Deployment Plan

1. Introduction & Project Overview **2**
1.1 Application Overview 2
1.2 Purpose of This Document 2
1.3 Intended Audience 2
2. Deployment Scope & Objectives **3**
2.1 Scope 3
2.2 Deployment Objectives 3
3. System Architecture **4**
3.1 High-Level Architecture 4
3.2 Data Flow Summary 4
4. Technical Stack & Dependencies **5**
4.1 Core Technologies 5
4.2 Hosting & Infrastructure 5
4.3 Scalability Architecture 6
5. Environment Strategy **7**
5.1 Environment Variable Matrix 7
6. CI/CD Pipeline & Automation **8**
6.1 Pipeline Overview 8
6.2 PR Preview Deployments 8
6.3 Automated Build Configuration 8
7. Deployment Procedure **9**
8. Integration Security & Credential Management **10**
8.1 API Key Architecture 10
8.2 Supabase Row Level Securityf (RLS) 10
8.3 Input Validation with Zod 10
8.4 HTTPS Everywhere 10
9. Post-Deployment Verification & Testing **11**
10. Rollback Plan **12**
11. Monitoring & Incident Response **13**
11.1 Application Monitoring 13
11.2 Alerting Thresholds 13
12. Known Limitations & Future Roadmap **14**
12.1 Current Limitations (v1.0) 14
12.2 Planned Enhancements (v2.0) 14

1. Introduction & Project Overview

1.1 Application Overview

- BizAssist AI Hub is an AI-powered Customer Relationship Management (CRM) and business operations platform designed specifically for Small and Medium Enterprises (SMEs) in Malaysia. The platform consolidates multi-channel customer communications, transforms unstructured social media (e.g. Whatsapp, Messenger) chat data into structured business insights, and enables automated order management — all powered by the Google Gemini large language model.

1.2 Purpose of This Document

- This Deployment Plan outlines the technical strategy, processes, and procedures required to successfully deploy BizAssist AI Hub from a development environment into a production-grade hosting infrastructure. It serves as the authoritative reference for all deployment, release, and operational activities.

1.3 Intended Audience

- Software Development Team
- DevOps / Platform Engineers
- Technical Evaluators and Hackathon Judges
- Project Stakeholders

2. Deployment Scope & Objectives

2.1 Scope

This deployment covers the following components of the BizAssist AI Hub platform:

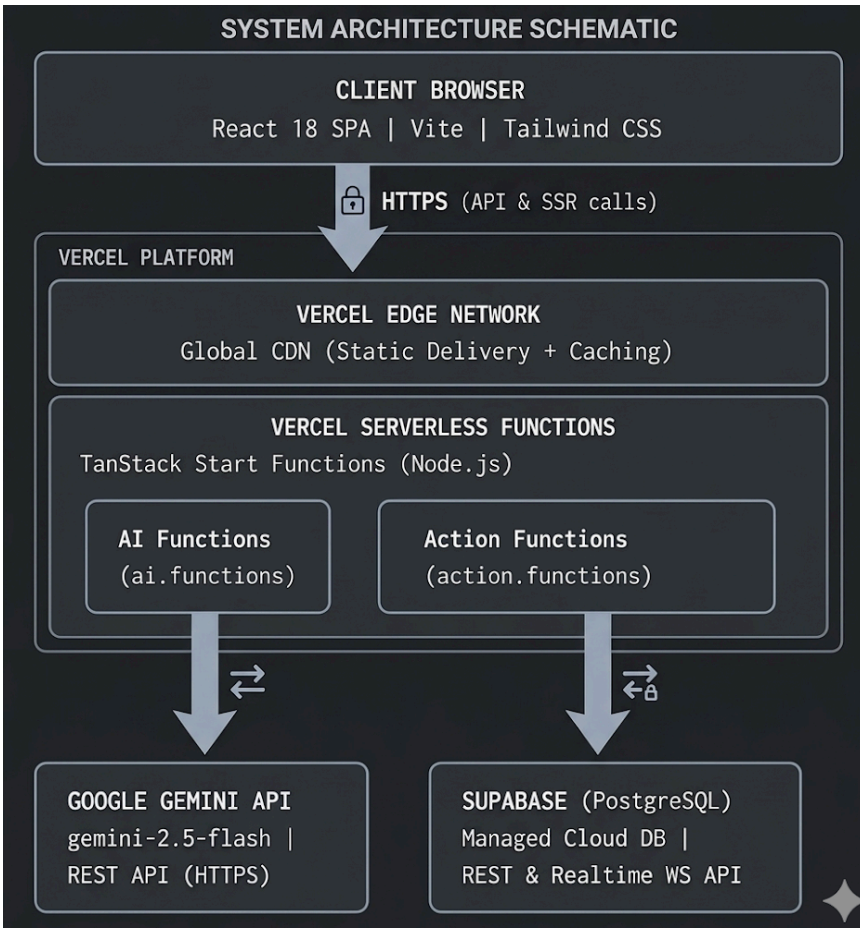
Component	Description	Type
Frontend Web Application	React SPA with full UI, routing, and state management	Static + Edge
API / Server Functions	TanStack Start serverless functions handling AI calls and database writes	Serverless
Database	Supabase PostgreSQL instance for storing orders, meetings, customers, and chat threads	Managed DBaaS
AI Engine Integration	Real-time calls to Google Gemini 2.5 Flash for conversation analysis and action generation	Third-Party API

2.2 Deployment Objectives

1. Deploy a fully functional, production-ready web application accessible via a public URL.
2. Ensure all third-party API integrations (Gemini AI, Supabase) are operational in the production environment.
3. Establish a repeatable, automated deployment pipeline for future releases.
4. Enforce security best practices for credential management and data access.
5. Achieve zero manual configuration needed for future redeployments.

3. System Architecture

3.1 High-Level Architecture



3.2 Data Flow Summary

1. The user's browser loads the compiled React application from the Vercel CDN.
2. When the user triggers an AI feature (e.g., "Run Analysis"), the browser calls a TanStack server function endpoint.
3. The server function securely retrieves the Gemini API key from environment variables and forwards the request to the Google Gemini REST API.
4. The AI response is parsed, validated, and returned to the browser.
5. When the user confirms a business action (e.g., an order), the server function uses the Supabase Service Role Key to perform a privileged write operation to the PostgreSQL database.

4. Technical Stack & Dependencies

4.1 Core Technologies

Layer	Technology	Version	Purpose
UI Framework	React	18.x	Component-based frontend rendering
Build Tool	Vite	5.x	Fast development server & optimized production bundling
Routing / Backend	TanStack Start	Latest	File-based routing + server functions
Styling	Tailwind CSS	3.x	Utility-first CSS styling
Language	TypeScript	5.x	Type-safe development across the full stack
Database	Supabase	2.x	Managed PostgreSQL with real-time and REST APIs
AI Integration	Google Gemini	2.5 Flash	LLM for chat analysis and action planning
Input Validation	Zod	3.x	Runtime type safety for API payloads

4.2 Hosting & Infrastructure

Service	Provider	Purpose
Web Hosting	Vercel	Static frontend + serverless API functions
Database	Supabase	Managed PostgreSQL with Row Level Security
AI API	Google AI Studio	Gemini LLM inference endpoint
DNS / SSL	Vercel (managed)	Automated HTTPS certificate via Let's Encrypt
Version Control	GitHub	Source code management and CI/CD trigger

4.3 Scalability Architecture

BizAssist is architected on a serverless-first model. This provides the following scalability properties:

- **Auto-Scaling:** Vercel's serverless functions scale from 0 to thousands of concurrent invocations automatically. No manual server provisioning is required.
- **Global Distribution:** Static assets (HTML, JS, CSS) are distributed across Vercel's global Edge Network, ensuring low latency for users regardless of geographic location.
- **Database Scalability:** Supabase's managed PostgreSQL instance can be independently scaled (vertically or horizontally via read replicas) without any changes to the application code.

5. Environment Strategy

BizAssist maintains three isolated environments to support safe development, quality assurance, and production releases.

Environment	Branch	URL	Database	Purpose
Development	`feature/*`	`localhost:5173`	Local / Seeded Mock Data	Active development and local testing
Preview / Staging	`develop`	Auto-generated Vercel preview URL	Staging Supabase Project	QA, integration testing, stakeholder review
Production	`main`	`bizassist.vercel.app`	Production Supabase Project	Live end-user access

5.1 Environment Variable Matrix

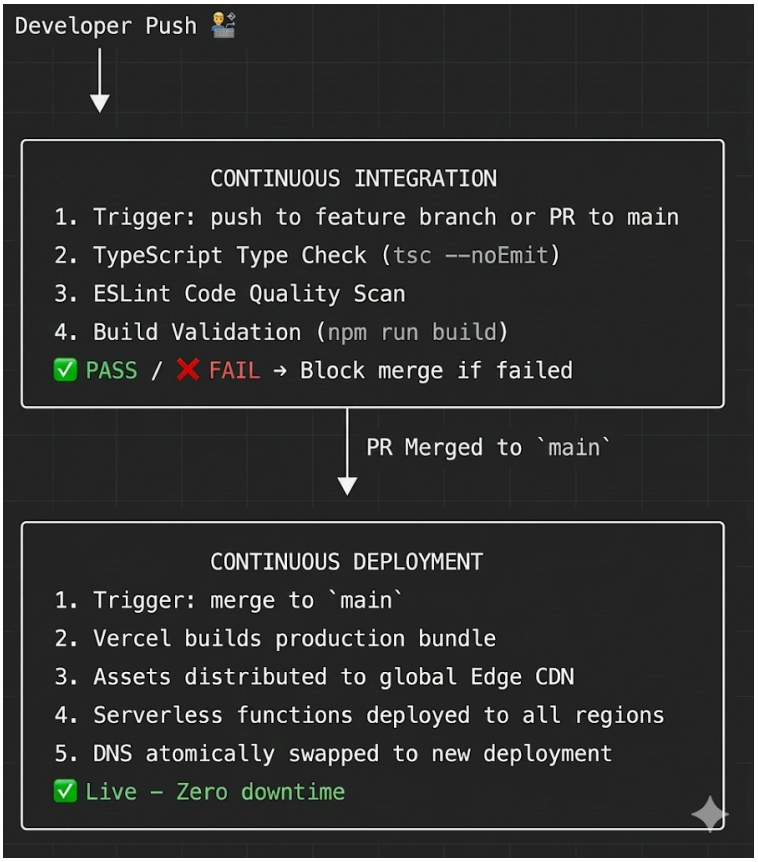
All credentials are scoped per environment and injected at build time via the hosting platform's secrets manager.

Variable	Development	Preview	Production	Purpose
`VITE_GEMINI_API_KEY`	Dev key	Test key	Production key	Gemini API Authentication
`VITE_SUPABASE_URL`	Local URL	Staging URL	Production URL	Database connection
`VITE_SUPABASE_ANON_KEY`	Dev anon key	Staging anon key	Production anon key	Public read queries
`SUPABASE_SERVICE_ROLE_KEY`	Dev service key	Staging service key	Production service key	Privileged server-side writes

6. CI/CD Pipeline & Automation

BizAssist implements a fully automated Continuous Integration and Continuous Deployment pipeline using GitHub Actions and Vercel's native Git integration.

6.1 Pipeline Overview



6.2 PR Preview Deployments

Every Pull Request automatically receives a unique, isolated preview URL (e.g., bizassist-git-feature-xyz.vercel.app). This allows QA reviewers and stakeholders to validate UI changes interactively in a live environment before merging into production.

6.3 Automated Build Configuration

The following vercel.json configuration is used to define the build and routing behavior:

json

```
{
 "buildCommand": "npm run build",
 "outputDirectory": "dist",
 "installCommand": "npm install",
 "framework": "vite"
}
```

7. Deployment Procedure

This section outlines the step-by-step procedure for performing the first-time production deployment.

Step 1: Database Migration

1. Log into the Supabase Dashboard at https://app.supabase.com.
2. Create a new project for the production environment.
3. Navigate to SQL Editor and execute the database schema script located at /supabase/schema.sql in the repository. This creates all necessary tables: organizations, customers, orders, order_items, meetings, chat_threads.
4. Navigate to Authentication > Row Level Security and verify that RLS is enabled on all tables.
5. Copy the Project URL, Anon Key, and Service Role Key from the Supabase project settings.

Step 2: Vercel Project Setup

1. Log into https://vercel.com and click Add New > Project.
2. Connect to the GitHub repository bizassist-ai-hub and authorize Vercel access.
3. Vercel will auto-detect the Vite framework. Confirm the default build settings.

Step 3: Inject Environment Variables

1. In the Vercel project settings, navigate to Settings > Environment Variables.
2. Add all four required environment variables listed in Section 5.1 under the Production scope.
3. Ensure SUPABASE_SERVICE_ROLE_KEY is scoped to Production only, not Preview.

Step 4: Trigger Deployment

1. Click Deploy on the Vercel dashboard, or push a commit to the main branch.
2. Monitor the build logs in real-time within the Vercel dashboard.
3. A successful build will display a green checkmark and a production URL.

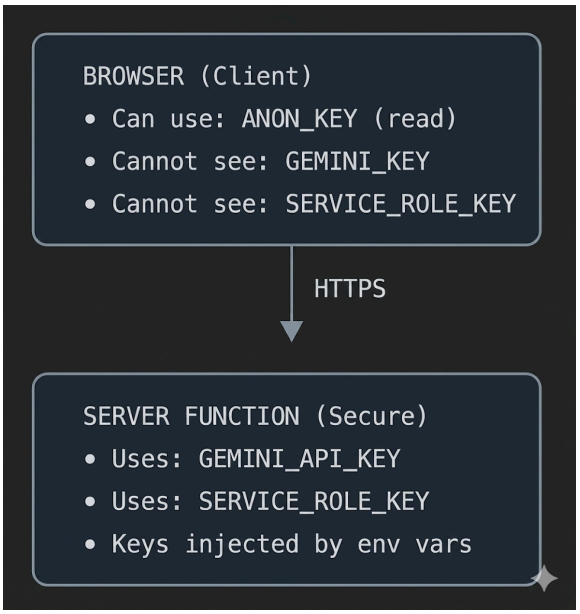
Step 5: Validate Deployment

Proceed to Section 9 (Post-Deployment Verification) to confirm all integrations are functional.

8. Integration Security & Credential Management

8.1 API Key Architecture

BizAssist follows a **zero-trust credential model**, ensuring no sensitive keys are ever exposed to the client browser.



8.2 Supabase Row Level Security (RLS)

All database tables have Row Level Security enabled. Each record is scoped to an organization_id, meaning that even if the Anon Key is compromised, it is structurally impossible for one organization's data to leak to another tenant.

8.3 Input Validation with Zod

All inputs flowing from the client browser to the server, and all outputs from the AI engine (Gemini), are validated against strict Zod schemas before being processed or persisted to the database. This prevents:

- **SQL Injection** via malformed user inputs.
- **Data Corruption** from AI model hallucinations producing unexpected data types.
- **Type Coercion Exploits** by enforcing strict TypeScript type constraints at runtime.

8.4 HTTPS Everywhere

All communication between the browser, Vercel serverless functions, Supabase, and the Gemini API occurs exclusively over HTTPS. SSL/TLS certificates for the Vercel-hosted domain are automatically issued and renewed by Let's Encrypt at no cost.

9. Post-Deployment Verification & Testing

After deployment, the following System Integration Tests (SIT) must be executed and documented.

Test ID	Test Case	Expected Result	Status
SIT-01	Navigate to production URL	Application loads without errors	Pass / Fail
SIT-02	Open Dashboard page	KPI cards and charts render with data	Pass / Fail
SIT-03	Open Messages Inbox	Chat threads are visible and selectable	Pass / Fail
SIT-04	Click "Run Analysis" on a chat	AI-generated analysis summary appears	Pass / Fail
SIT-05	Click "Generate Actions" on a chat	Action proposals list is rendered	Pass / Fail
SIT-06	Click "Confirm" on an Order proposal	Invoice modal appears and order is saved	Pass / Fail
SIT-07	Navigate to Meetings page	Calendar renders and meetings are displayed	Pass / Fail
SIT-08	Navigate to Analytics page	Charts render with correct data	Pass / Fail
SIT-09	Toggle dark/light theme	Theme switches without page reload	Pass / Fail
SIT-10	Verify HTTPS certificate	Browser padlock icon present, no SSL warnings	Pass / Fail

10. Rollback Plan

In the event of a critical failure following a production deployment, the following rollback procedure must be initiated within 15 minutes.

Immediate Rollback (< 5 minutes)

1. Navigate to the Vercel Dashboard.
2. Select the Deployments tab on the project.
3. Locate the last known stable deployment (status: READY).
4. Click ··· (Options) > Promote to Production.
5. Vercel will atomically re-route all production traffic to the previous deployment with zero downtime.

Database Rollback (if schema migration was applied)

1. If a destructive database migration was executed as part of the release, restore the pre-migration state from the Supabase Point-in-Time Recovery (PITR) backup (available on paid Supabase plans).
2. The team should maintain a rollback.sql script alongside every migrate.sql script for safe, reversible schema changes.

11. Monitoring & Incident Response

11.1 Application Monitoring

Tool	Purpose
Vercel Analytics	Tracks real user performance metrics, function invocation counts, and error rates
Supabase Dashboard	Monitors database connection counts, query latency, and storage usage
Vercel Function Logs	Real-time streaming logs from all serverless API functions for debugging
Google AI Studio	Monitors Gemini API token consumption and rate limit usage

11.2 Alerting Thresholds

- **API Error Rate > 5%:** Triggers internal team alert for investigation.
- **Database Connection Pool Saturation > 80%**: Indicates need to upgrade Supabase plan or enable Supavisor pooling.
- **Gemini API Rate Limit Warnings:** Triggers a review of user activity patterns to implement request caching.

12. Known Limitations & Future Roadmap

12.1 Current Limitations (v1.0)

Limitation	Description	Mitigation
Gemini Free Tier Rate Limit	Free tier allows 15 requests/minute. High-speed repeated clicks will trigger a temporary 429 error.	Upgrade to paid Gemini API tier for production usage
Supabase RLS (Demo Mode)	The demo org uses a shared "demo" slug for evaluation purposes.	Multi-tenant RLS policies to be finalized before commercial launch
PDF Generation	Invoice PDF rendering is currently a UI prototype (visual only).	Integrate `@react-pdf/renderer` or a Puppeteer-based serverless function

12.2 Planned Enhancements (v2.0)

1. **WhatsApp Business API Integration:** Connect directly to the Meta WhatsApp Cloud API for live, real-time message receiving and sending without manual chat exports.
2. **Background Job Queues:** Migrate long-running AI batch jobs to a dedicated queue system (e.g., Inngest or AWS SQS) to prevent serverless function timeout constraints.
3. **Multi-Tenancy & Authentication:** Implement Supabase Auth for multi-user access, role-based permissions (Admin, Agent, Read-Only), and organization-level data isolation.
4. **Redis Edge Cache:** Cache Gemini AI analysis results for identical conversation hashes to reduce API costs by an estimated 60-70%.